

Advanced Python

Comprehensions, Generators, map/filter/reduce, Context Managers & Regular Expressions

Session 12

What We're Covering Today

- Comprehensions — building lists, dicts, and sets in a single readable line
- Generators — producing values one at a time, without holding everything in memory
- map(), filter(), and reduce() — the functional-style alternative to comprehensions
- Context managers — what with is really doing, and how to build your own
- Regular expressions — finding and validating patterns inside text
- Today's tasks and two mini projects combining everything



PART 1

Comprehensions

A single, readable line that replaces a full for-loop for building a list, dict, or set

What Is a Comprehension?



A loop, compressed to one line

- A comprehension builds a new list, dictionary, or set by describing what each item should be, directly inside the brackets
- It’s not a new concept — it’s a more compact, more readable way to write a for-loop that builds up a collection
- squares = [x*x for x in range(5)] reads almost like English: "x times x, for every x in range(5)"
- This is one of the most distinctly "Pythonic" habits — experienced Python code leans on comprehensions constantly



List Comprehension — The Basic Pattern

- The pattern is always [expression for item in iterable] — expression decides what goes into the new list
- Compare the loop version and the comprehension version below — they produce exactly the same result
- The comprehension is a single expression, not a series of statements — it can be used directly wherever a list is needed

```
# The loop version
squares = []
for x in range(5):
    squares.append(x * x)
# [0, 1, 4, 9, 16]

# The comprehension version — same result, one line
squares = [x * x for x in range(5)]
# [0, 1, 4, 9, 16]
```



Adding a Condition — Filtering While You Build

- Adding if at the end filters which items get included — only items where the condition is True make it into the result
- This replaces a loop with a for AND an if inside it, combining both into the same single line
- The condition always comes after the for clause, unlike a ternary if/else, which comes before the for

```
# Only even squares
evens_squared = [x * x for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]

# if/else BEFORE the for — a value for every item, not a filter
labels = ["even" if x % 2 == 0 else "odd" for x in range(5)]
# ['even', 'odd', 'even', 'odd', 'even']
```

Dict Comprehension

- The same idea works for dictionaries — the pattern becomes {key_expr: value_expr for item in iterable}
- This is extremely common for transforming one dictionary into another, or building a dictionary from two lists
- zip() pairs up two sequences item by item — exactly what's needed to build a dict from separate keys and values

```
names = ["Ahmed", "Sara", "Ali"]
ages = [25, 22, 28]

people = {name: age for name, age in zip(names, ages)}
# {'Ahmed': 25, 'Sara': 22, 'Ali': 28}

prices = {"pen": 20, "book": 150, "bag": 800}
discounted = {item: price * 0.9 for item, price in
prices.items()}
# {'pen': 18.0, 'book': 135.0, 'bag': 720.0}
```

Set Comprehension

- Curly braces with no colon build a set instead of a dict — {expr for item in iterable}
- A set comprehension automatically removes duplicates, exactly like any other set
- Useful whenever you need unique results and don’t care about order — like collecting every distinct value in a dataset

```
words = ["apple", "banana", "apple", "cherry", "banana"]

unique_lengths = {len(w) for w in words}
# {5, 6}   — duplicate lengths collapsed automatically

first_letters = {w[0].upper() for w in words}
# {'A', 'B', 'C'}
```

Nested Comprehensions — Use Sparingly



Powerful, but read twice

- A comprehension can contain another comprehension — most often used to flatten a list of lists into one flat list
- [num for row in matrix for num in row] reads left to right: for every row in matrix, for every num in that row, keep num
- This is genuinely useful, but readability drops fast past two levels — a normal nested loop is often clearer at that point
- The same "would this be clearer written out?" test from lambda applies here too

Practice Set — Comprehensions

Try these yourself first — no code shown, that's the point

1. Write a list comprehension that returns the cubes of numbers 1 through 10

2. Write a list comprehension that keeps only the words longer than 4 letters from a sentence

3. Write a dict comprehension that maps each word in a list to its length

4. Write a set comprehension that returns every unique vowel found in a sentence

5. Flatten a list of lists like [[1,2], [3,4], [5]] into a single flat list using a nested comprehension



PART 2

Generators

What if you never needed to hold the whole result in memory at once?

What Is a Generator?



One value at a time, on demand

- A generator is a special kind of function that produces a sequence of values one at a time, instead of building and returning the whole thing at once
- A normal function computes everything, then hands it all back with return — a generator pauses after each value, and only continues when asked for the next one
- This matters enormously for big or even infinite sequences — you can’t hold "every number forever" in a list, but you can generate them one at a time
- Under the hood, this is exactly how range() and file objects already work — you’ve been using generator-like objects since your very first loop

yield vs. return

- A function containing yield anywhere in its body automatically becomes a generator function — calling it doesn't run it, it creates a generator object
- return ends a function immediately and hands back one final value — yield pauses the function, hands back one value, and remembers exactly where it left off
- The next time the generator is asked for a value, execution resumes right after that yield, not from the top

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count     # pause here, hand back "count"
        count += 1

counter = count_up_to(3)
print(counter)        # <generator object ...>  — nothing run yet
```



Pulling Values Out with next()

- next() asks a generator to run until its next yield, and returns that value
- Each call to next() resumes exactly where the last one paused — the generator remembers its own internal state
- When there's nothing left to yield, Python raises StopIteration — this is exactly what a for loop is silently catching for you

```
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

counter = count_up_to(3)
next(counter)  # 1
next(counter)  # 2
next(counter)  # 3
next(counter)  # StopIteration — nothing left

for num in count_up_to(3):  # handles next()/StopIteration for you
    print(num)              # 1, 2, 3
```

Generator Expressions — Comprehension Syntax, Lazy Result

- Swapping a list comprehension’s square brackets for parentheses turns it into a generator expression
- It looks almost identical to a list comprehension, but produces values lazily instead of building the whole list immediately
- This is the fastest way to get generator behavior when you don’t need a full yield-based function

```
squares_list = [x * x for x in range(1_000_000)]     # built NOW, all at once
squares_gen  = (x * x for x in range(1_000_000))     # built lazily, one at a time

print(type(squares_gen))   # <class 'generator'>

total = sum(x * x for x in range(1_000_000))   # never stores the full list
```

Lazy Evaluation, Visualized

[x*x for x in range(5)] — a list



All 5 values computed and held in memory immediately — before you even use one of them

(x*x for x in range(5)) — a generator



Only ONE value exists at a time — the rest aren't computed until you ask via next()

Why this matters: A list of 10 million squared numbers holds all 10 million in memory at once. A generator holds exactly one value at a time — the other 9,999,999 simply don't exist yet.

This is called lazy evaluation — values are produced only the moment they're actually needed, never before.

Real-World Use: Reading a Huge File Line by Line

- A file object is itself already a generator-like object — looping over it never loads the whole file into memory
- For a genuinely massive log file, readlines() could crash a machine — iterating with a generator never will, no matter the file size
- This is the exact same "one value at a time" idea, applied to something you've already been doing since file handling

```
def read_large_file(filename):
    with open(filename) as file:
        for line in file:     # one line at a time, never the whole file
            yield line.strip()

for line in read_large_file("huge_log.txt"):
    if "ERROR" in line:
        print(line)
```

Practice Set — Generators

Try these yourself first — no code shown, that's the point

1. Write a generator function even_numbers(n) that yields every even number from 0 up to n

2. Manually call next() on it three times, then use a for loop to consume the rest

3. Rewrite a list comprehension you wrote earlier this session as a generator expression

4. Write a generator that yields Fibonacci numbers one at a time, forever — with no upper limit



PART 3

map(), filter(), and reduce()

A different, more functional style for transforming and combining sequences

map() — Transform Every Item

- map(function, iterable) applies function to every item and returns a lazy map object — wrap it in list() to see all the results at once
- map() can also take more than one iterable at once — the function is called with one item from each, pairwise
- You met map() with lambda already — it works exactly the same with a named function too

```
nums = [1, 2, 3, 4]
doubled = list(map(lambda n: n * 2, nums))
# [2, 4, 6, 8]

# map() over TWO iterables at once
a = [1, 2, 3]
b = [10, 20, 30]
sums = list(map(lambda x, y: x + y, a, b))
# [11, 22, 33]
```



filter() — Keep Only What Passes

- filter(function, iterable) keeps only the items where function returns True — also lazy, also needs list() to see results
- Passing None as the function keeps only the items that are already truthy — a quick way to drop empty strings, zeros, and Nones from a list
- filter() is a direct alternative to a list comprehension with just an if clause and no transformation

```
nums = [0, 1, 2, "", "hello", None, 3, False]

evens = list(filter(lambda n: isinstance(n, int) and n % 2 == 0, nums))
# [0, 2]

truthy_only = list(filter(None, nums))
# [1, 2, 'hello', 3]
```



reduce() — Combine Everything Into One Value

- reduce(), from the functools module, repeatedly combines items two at a time until only one value is left
- It takes a function of two arguments — the "running total so far" and "the next item" — and keeps folding them together
- Unlike map() and filter(), reduce() isn’t built in by default — it has to be imported, a deliberate signal that it’s used far less often

```
from functools import reduce

nums = [1, 2, 3, 4]
total = reduce(lambda acc, n: acc + n, nums)
# 10  — same as ((1 + 2) + 3) + 4

product = reduce(lambda acc, n: acc * n, nums)
# 24  — same as ((1 * 2) * 3) * 4
```

Comprehensions vs. map()/filter() — The Pythonic Take

Prefer Comprehensions

- Most experienced Python developers reach for a comprehension by default — it's considered more readable
- [x*x for x in nums if x > 0] reads as one clear thought; the equivalent map+filter needs two nested calls
- This is the standard, idiomatic choice for transforming and filtering in Python

map()/filter() Still Matter

- You'll see them constantly in real, existing codebases — recognizing them fluently matters even if you don't reach for them first
- They shine when you already have a named function, and don't want to wrap it in a comprehension's syntax
- reduce() specifically has no comprehension equivalent — it fills a real, different gap

Practice Set — map(), filter(), reduce()

Try these yourself first — no code shown, that's the point

1. Use map() to convert a list of strings into a list of their lengths

2. Use filter() to keep only the strings that start with a vowel

3. Use reduce() to find the maximum value in a list, without using max()

4. Rewrite one of the map()/filter() tasks above as a single list comprehension, and compare readability



PART 4

Context Managers

You’ve used with open(…) for weeks — now let’s see exactly what it’s doing, and build your own

What Is a Context Manager?



Guaranteed setup and cleanup

- A context manager is any object that defines what happens right when a with block starts, and what happens right when it ends — no matter how it ends
- with open("file") as f: is the context manager you already know — it opens the file on entry, and guarantees it closes on exit, even if an error occurs
- The same guaranteed-cleanup pattern applies to database connections, network sockets, locks in multi-threaded code, and timers
- Anything with a clear "start" and a "must always clean up afterward" shape is a candidate for a context manager



Writing Your Own — __enter__ and __exit__

- __enter__ runs when the with block starts — whatever it returns becomes the as variable
- __exit__ runs when the block ends, whether it finished normally or an exception was raised inside it
- This is the exact same dunder-method pattern from OOP — with is just Python calling these two methods automatically for you

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        import time
        print(f"Took {time.time() - self.start:.4f} seconds")

with Timer():
    total = sum(range(1_000_000))
# Took 0.0231 seconds
```



@contextmanager — A Simpler Way to Write One

- contextlib's @contextmanager decorator lets you write a context manager as a single generator function, instead of a full class
- Code before yield runs on entry; code after yield runs on exit — yield itself marks the exact point where the with block's code runs
- For simple cases, this is noticeably less boilerplate than writing __enter__ and __exit__ by hand

```
from contextlib import contextmanager
import time

@contextmanager
def timer():
    start = time.time()
    yield                    # the with block runs here
    print(f"Took {time.time() - start:.4f} seconds")

with timer():
    total = sum(range(1_000_000))
```

Practice Set — Context Managers

Try these yourself first — no code shown, that's the point

1. Write a class-based context manager that prints "Starting..." on entry and "Finished." on exit

2. Rewrite the same context manager using @contextmanager instead of a class

3. Use your context manager to wrap a block of code, and confirm the exit message prints even when the block raises an error



PART 5

Regular Expressions

A mini-language for describing patterns in text — built into Python through the re module

What Is a Regular Expression?



A pattern, not a literal string

- A regular expression (regex) describes a pattern of text, rather than one exact string — "a digit followed by three letters" instead of "5abc"
- This lets you validate formats (emails, phone numbers), search through huge amounts of text, and extract specific pieces from messy strings
- Python’s built-in re module provides everything needed to work with regex — no installation required
- Regex has its own compact syntax that looks unfamiliar at first, but a handful of building blocks cover the vast majority of real use



The Core Functions: search(), match(), findall()

- re.search() looks for the pattern anywhere in the string, and returns the first match it finds, or None
- re.match() only checks starting at the very beginning of the string — a match anywhere else doesn’t count
- re.findall() returns every match in the string as a list, not just the first one

```
import re

text = "Contact us at 0300-1234567 or 0321-9876543"

re.search(r"\d{4}-\d{7}", text)    # matches "0300-1234567"
re.match(r"Contact", text)         # matches — string starts with it
re.match(r"0300", text)            # None — string doesn't START with it
re.findall(r"\d{4}-\d{7}", text)   # ['0300-1234567', '0321-9876543']
```

Building Blocks — The Patterns You'll Use Constantly



\d \w \s

Any digit, any word character (letter/digit/underscore), any whitespace



*** + ?**

0 or more, 1 or more, and 0 or 1 of whatever came right before it



{n} {n,m}

Exactly n repeats, or between n and m repeats



^ $

Anchors — the very start and the very end of the string



( ) [ ]

A capture group, and a character class — "any one of these characters"

Anatomy of a Pattern

```
r'(\d{3})-(\d{4})'
```

( ) — A capture group — pulls out just this piece of the match

\d — Any single digit, 0–9

{3} / {4} — Exactly that many repeats of what came before it

- — A literal character — matches an actual dash, nothing special

This pattern matches phone extensions like "123-4567" and captures "123" and "4567" separately

Groups and re.sub() — Extracting and Replacing

- Parentheses in a pattern create capture groups — .groups() on a match gives you each captured piece separately
- re.sub(pattern, replacement, text) finds every match and replaces it — exactly like find-and-replace, but pattern-based instead of exact-text-based
- This combination — match, capture, replace — is the foundation of most real-world text-cleaning code

```
import re

match = re.search(r"(\d{3})-(\d{4})", "Ext: 123-4567")
match.group()    # '123-4567'  — the whole match
match.group(1)   # '123'       — first capture group
match.group(2)   # '4567'      — second capture group

re.sub(r"\d", "*", "Call 0300-1234567")
# 'Call ****-*******'
```



Real-World Use: Validating and Extracting

- A well-formed regex is the standard way to validate whether input roughly matches an expected format, like an email address
- re.fullmatch() requires the ENTIRE string to match, not just some part of it — exactly what validation needs
- This exact pattern — fullmatch for validation, findall/sub for extraction and cleanup — covers most real-world text-processing needs

```
import re

def is_valid_email(email):
    pattern = r"[\w.]+@[\w]+\.[a-z]{2,}"
    return bool(re.fullmatch(pattern, email))

is_valid_email("ahmed@example.com")   # True
is_valid_email("not-an-email")        # False
```

Practice Set — Regular Expressions

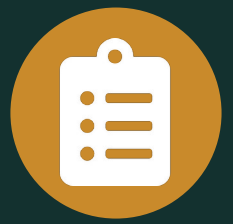
Try these yourself first — no code shown, that's the point

1. Write a regex that finds every number inside a sentence, using re.findall()

2. Write a regex that validates a Pakistani-style phone number in the format 03XX-XXXXXXX

3. Use re.sub() to replace every vowel in a sentence with an asterisk

4. Use a capture group to extract just the domain name from an email address



INDUSTRY-STYLE TASKS

Build the Things You'll Actually Be Asked to Build

Not toy examples — lazy pipelines, safe resource handling, and pattern validation are everyday, real-world work

Today's Tasks — Part A: Comprehensions & Generators

Small tasks — do them in order

1. Given a list of product dictionaries, use a dict comprehension to build {name: price} for every product under 1000.

2. Write a generator function batch(iterable, size) that yields successive chunks of a given size from a list.

3. Rewrite a for-loop you wrote in an earlier session as either a comprehension or a generator expression, whichever fits better.

Today's Tasks — Part B: map/filter/reduce & Context Managers

Small tasks — do them in order

1. Use reduce() to concatenate a list of strings into one, separated by commas, without using .join().

2. Write a class-based context manager that opens a file, and automatically writes a "closed at [time]" log line on exit.

3. Use map() and filter() together to get the uppercase versions of every word longer than 3 letters in a sentence.

Today's Tasks — Part C: Regular Expressions

Small tasks — do them in order

1. Write a function that extracts every hashtag (e.g. #python) from a block of social media text using re.findall().

2. Write a password_strength(password) function using regex to check for at least one digit, one uppercase letter, and a minimum length.

3. Use re.sub() to normalize messy whitespace in a paragraph — collapse multiple spaces into exactly one.



PART I

Put It All Together

Two projects combining comprehensions, generators, functional tools, context managers, and regex

Mini Project 1: Log File Analyzer



Generators + regex + context managers

- Write a generator function that reads a log file line by line, never loading the whole file into memory at once
- Use regex to extract the timestamp, severity level, and message from each line, based on a consistent log format
- Use a context manager to time the entire analysis and guarantee the file is properly closed, even on error
- Report the count of ERROR vs WARNING vs INFO lines using a dict comprehension over the results

Mini Project 2: Sales Data Pipeline



Comprehensions + map/filter/ reduce, combined

- Given a list of sale dictionaries, use a list comprehension to filter only sales above a certain amount
- Use map() to apply a tax calculation across every remaining sale
- Use reduce() to compute the total revenue in a single fold, without a manual loop
- This mirrors how real data-cleaning and reporting pipelines are structured before reaching for a library like pandas



RECAP

Comprehensions, generators, functional tools, context managers, and regex are the difference between code that works and code that reads like a professional wrote it.

Next up: Git & GitHub, in depth — branches, merging, pull requests, and resolving conflicts